

Analysis of RNA-Seq Data Using TopHat and Cufflinks

Sreya Ghosh and Chon-Kit Kenneth Chan

Abstract

The recent advances in high throughput RNA sequencing (RNA-Seq) have generated huge amounts of data in a very short span of time for a single sample. These data have required the parallel advancement of computing tools to organize and interpret them meaningfully in terms of biological implications, at the same time using minimum computing resources to reduce computation costs. Here we describe the method of analyzing RNA-seq data using the set of open source software programs of the Tuxedo suite: TopHat and Cufflinks. TopHat is designed to align RNA-seq reads to a reference genome, while Cufflinks assembles these mapped reads into possible transcripts and then generates a final transcriptome assembly. Cufflinks also includes Cuffdiff, which accepts the reads assembled from two or more biological conditions and analyzes their differential expression of genes and transcripts, thus aiding in the investigation of their transcriptional and post transcriptional regulation under different conditions. We also describe the use of an accessory tool called CummeRbund, which processes the output files of Cuffdiff and gives an output of publication quality plots and figures of the user's choice. We demonstrate the effectiveness of the Tuxedo suite by analyzing RNA-Seq datasets of *Arabidopsis thaliana* root subjected to two different conditions.

Key words RNA-seq, Bowtie, TopHat, Cufflinks, Cuffmerge, Cuffcompare, Cuffdiff, CummeRbund, Differential gene expression, Transcriptome assembly

1 Introduction

In the early years of transcriptome studies, microarray technologies based on nucleic acid hybridization were predominately used for gene expression analysis. However, these technologies posed a limitation in quantifying the various kinds of RNA molecules that are expressed by the genome under myriad biological conditions at different points of time. The limitation is mainly due to the reliance on extensive prior knowledge of the genome [1]. The recent advances in Next Generation Sequencing (NGS) of DNA [2] have led to a burst in RNA analysis through massively parallel cDNA sequencing that has accelerated our understanding of the dynamics of genome expression and genome structural variation [1, 3, 4]. Methods have also been devised to directly sequence RNA on a massively parallel scale without intermediate cDNA generation [5]

in order to avoid some of the pitfalls or artifacts produced during cDNA synthesis from RNA strands [6], producing reads which are bias free and more comprehensive. High throughput RNA-seq produces large volumes of data in a single experiment, which need to be assembled correctly in order to interpret them meaningfully. A large number of computation methods and tools have been developed for efficient assembly, quantification, and differential analysis of such data [7]. Of these the “Tuxedo suite” of programs—Bowtie, TopHat, and Cufflinks—along with accessory tools and programs, like CummeRbund, have gained a lot of popularity. The TopHat algorithm maps sequenced reads to a reference genome without relying on prior annotation of genes and accounts for alternative splicing of a primary transcript, while the Cufflinks algorithm estimates transcript abundances and differential expression. Analysis of large volumes of RNA-seq data with these algorithms has revealed novel isoforms and splice variants, as well as unannotated transcripts even in well studied biological models [8].

1.1 RNA-seq

The transcriptomic profile of an organism at any given time or condition gives the set of all its transcripts and their quantities present at the specific time point or condition. The transcriptome reveals a great deal about the functional aspects of the genome as well as the different kinds of biomolecules present within the cell or tissue. It is also very useful for studying the genetics behind growth, development and disease. Transcriptome studies aim at cataloging the different kinds of RNA in an organism, and quantifying the changes in expression levels of individual transcripts with time or under certain conditions [9]. Previously, there have been two approaches for sequencing RNA: (a) The hybridization based approach which involved the use of microarrays [10, 11] and (b) The sequencing approach which directly determined the cDNA sequence using Sanger Sequencing approaches. The latter approach developed as an alternative to microarray technology, because hybridization heavily relied upon known sequenced genes and genomic regions. As sequencing the whole cDNA segment to produce ESTs is expensive and has huge limitations when quantifying expressions, many tag based high throughput sequence approaches were developed for expression quantification. These included CAGE (Cap Analysis of Gene Expression) [12, 13], SAGE (Serial Analysis of Gene Expression) [14], and MPSS (Massively Parallel Signature Sequencing) [15, 16]. The advancement of high throughput NGS technology makes the RNA-seq technology feasible and affordable for transcriptome profiling. RNA-seq involves extracting the total RNA from a sample, then selecting an RNA population subset (such as poly A (+)), converting them to cDNA, and ligating the cDNA to specific adaptors at one or both ends of the fragment. These modified fragments are then sequenced from one (single end sequencing) or both ends (paired end sequencing), with or without amplification, in a high throughput NGS

sequencing machine. After sequencing, the RNA reads are mapped and quantified based on a reference genome [17]. If a reference genome is not available, de novo transcriptome assembly can be carried out [18]. The RNA-seq analysis is also good for determining spliced junctions with the resolution of a single base, determining the structural variations in the genome, and locating transcript boundaries and novel genes and transcripts [9]. However, there are weaknesses in the RNA-seq technology. For example, DNA artifacts can be synthesized during cDNA cloning and that could cause misinterpretations later on. This weakness can be remediated by carrying out RNA-seq in replicates to even out the differences or account for sample variability during analysis [6, 7]. For a comprehensive review of RNA-seq, it is recommended to review the paper by Wang et al. [9].

1.2 The Tuxedo Suite of Computational Tools

RNA-seq data are usually very large and require efficient algorithms which use minimum computing resources for their analysis.

There are three main steps in the reference-based RNA-seq analysis [7]:

1. Aligning RNA-seq reads to a reference genome or transcriptome.
2. Assembling the aligned reads into transcription units, or reconstruction of a transcriptome.
3. Performing differential expression of transcripts across different conditions or time points

Each of the three steps possesses different computation challenges [7]. Although there are several different algorithms and tools that address each of these challenges separately, in this chapter, we only illustrate the popular Tuxedo Suite which contains a comprehensive set of programs that perform reference-based RNA-seq analysis efficiently. The core programs for RNA-seq analysis in the Tuxedo suite are:

1. **Bowtie** [19], an ultrafast unspliced read aligner with high memory efficiency. It uses the Burrows Wheeler Transform [20] and FM indexing [21] to compact a reference genome into a data structure that is efficient for aligning short reads with perfect matches.
2. **TopHat** [22] uses an “exon first” approach to align reads. It uses Bowtie to map the short contiguous unspliced reads to a reference genome. The unmapped reads are then split into shorter fragments and then are aligned independently to identify splice junctions between exons.
3. **Cufflinks** [23] uses the alignment file from TopHat, or other aligners which allow spliced alignments, to assemble and reconstruct the transcriptome. It assembles the overlapping “bundles” of aligned reads into transcripts using a probabilistic

approach, then merges multiple conditions and estimates the transcript abundances using Cuffmerge and Cuffcompare respectively (*see* **Note 9**). It can also use **Cuffdiff** to calculate the differential gene expression.

4. Finally, an accessory tool, **CummeRbund** [24, 23], renders the Cuffdiff output into visual representations like bar, scatter and volcano plots.

The Tuxedo suite programs are all run in the command line. Alternatively, the web-based platform, Galaxy [25], provides a better user friendly interface to run the Tuxedo suite programs but the free public Galaxy server is limited to analysis of smaller datasets.

1.3 Other Tools and Comparison with Tuxedo

The Tuxedo suite provides a comprehensive workflow for RNA-seq analysis. There are a number of alternative packages and tools that can be used in conjunction with or as an alternative to any of the tools in the Tuxedo suite. For example there are many other unspliced read aligners like SHRiMP [26] and Stampy [27] that follow the seed method and Smith–Waterman extension to align unspliced reads to the genome with high accuracy, and spliced aligners like MapSplice [28] and GSNAP [29]. There are also transcriptome assemblers like Scripture [30] (which is similar to Cufflinks but uses a segmentation approach) and Velvet/Oases [31] (where Velvet assembles the reads into transcripts then Oases builds the transcript isoforms). Finally, transcript abundance can be calculated by alternative tools like ERANGE [17], and NEUMA [32] and differential expression using DEGseq [33] and DESeq [34]. Some of these tools work with minimal assumption about the library construction protocol but most RNA-seq analysis tools require paired-end reads. A good review of RNA-seq tools along with explanations of the algorithms can be found in Garber et al. [7].

The Tuxedo suite serves as a comprehensive transcriptome analysis solution and the algorithms are robust and accurate. They also have minimum computing requirements in terms of processor cores and memory compared to other programs. They have gained popularity and have been used in a number of high-profile transcriptome studies [35].

2 Materials

TopHat and Cufflinks are run using Linux command line and the CummeRbund program is run in an R interactive shell. In the following discussion, commands that were run in a Linux shell are shown with a '\$' prefix whereas those run in an R interactive shell are shown with a '>' prefix.

The software used require, for best results, a 64-bit version of the operating system, with at least 4 GB of RAM.

Before beginning the example run, it is recommended to create a working directory. Here we have used a directory called *exp_run*. Additionally we have downloaded all the tools and data in a *Downloads_tc* folder. All installation is carried out by creation of a *bin* directory in the user's home directory, which is added to the PATH environment variable. The pre-compiled executable binaries are added to this *bin* directory.

2.1 RNA-seq Data

```
$ mkdir exp_run
$ mkdir Downloads_tc
$ mkdir $HOME/bin
$ export PATH=$HOME/bin:$PATH
```

For the purpose of this demonstration, we have chosen an experiment on *Arabidopsis thaliana*. The raw RNA-seq data were obtained from the Sequence Read Archive (SRA) under the National Center for Biotechnology Information (NCBI) (<http://www.ncbi.nlm.nih.gov/sra>). The data are in the experiment with the series accession number GSE44062 in the Genome Expression Omnibus of the NCBI [36]. We have selected the first four sample runs (SRR671946, SRR671947, SRR671948, and SRR671949) for the purpose of this demonstration (see **Note 1**).

Thus, in this demonstration, we show the differential gene expression in *Arabidopsis thaliana* root subjected to two different conditions—one is treated with KNO₃, while the other with KCl as a control. There are two biological replicates for each condition (see **Note 2**).

The data are downloaded in the SRA format and converted to FASTQ using the SRA Toolkit (version 2.3.4-2). The precompiled toolkit package is downloaded, and the locations of executables are added to the PATH or the bin folder of the current user. Then the following command is executed to convert the zipped SRA file to FASTQ:

```
$ fastq-dump --split-3 --gzip --outdir ./ --define-qual '+' ./SRR000000.sra
```

Here, SRR000000 represents the file name.

These FASTQ data files are stored in the working directory, with their extensions changed from *.fq* to *.fastq*

```
$ cp SRR000000.fq $HOME/exp_run/SRR000000.fastq
```

2.2 Reference Genome

Bowtie and TopHat need a sequenced reference genome in order to align the raw RNA-seq reads. This genome has to be indexed before it is used. The Bowtie website has many pre-built indexes to be used for this purpose. Alternatively, Illumina's iGenome project provides Bowtie 1 and 2 indexes pre-built for a number of model organisms (<http://tophat.cbcb.umd.edu/igenomes.shtml>). In this demonstration, we shall use the *Arabidopsis thaliana* TAIR 10

package assembled by the iGenome project with sequences downloaded from Ensembl. Users have the freedom of building their own Bowtie indexes for organisms whose pre-built indexes are not available. One can do this using the build command from Bowtie.

First, you have to obtain the complete sequence file(s), then move it (or them) to the Bowtie2 unpacked directory, and type out the following command:

```
$ bowtie2-build [options] <comma_separated_list_of_FASTA_files> <user's_choice_of_base_name_of_bowtie2_index>
```

The output contains a set of new files with the user specified base name.

The iGenome package for Arabidopsis (Ensembl, TAIR 10) downloaded, when unpacked, appears as *Arabidopsis_thaliana*. Within it, there is an *Ensembl* directory and then a *TAIR10* directory. In the *TAIR10* directory there are three directories: *Annotation*, *GenomeStudio*, and *Sequence*. Of these the ones relevant to this analysis are *Annotation* and *Sequence*. The necessary files are copied out into the working directory:

```
$ cp Annotation/Genes/genes.gtf path-to-exp_run
$ cp Sequence/Bowtie2Index/genome.* path-to-exp_run
```

2.3 Software and Tools

2.3.1 Bowtie Software

This demonstration utilizes Bowtie2-2.1.0 which can be downloaded from the Bowtie website (<http://sourceforge.net/projects/bowtie-bio/files/bowtie2/2.1.0/>). Download the binaries for Linux 64-bit system to the *Downloads_tc* folder as a tarball and unpack them.

```
$ unzip bowtie2-2.1.0-linux-x86_64.zip
```

From within the unpacked bowtie2 directory, copy out the executable binaries into the bin folder:

```
$ cd bowtie2-2.1.0
$ cp * path-to-USER/bin
```

2.3.2 TopHat Software

Download the Linux 64-bit binary version of Tohat-2.0.10 from the TopHat website (<http://tophat.cbcb.umd.edu/index.shtml>) to the *Downloads_tc* folder as a tarball, and unpack them.

```
$ tar -xvzf tophat-2.0.10.Linux_x86_64.tar.gz
```

From within the unpacked TopHat directory, copy out the executable binaries into the bin folder:

```
$ cd tophat-2.0.10.Linux_x86_64
$ cp * path-to- USER/bin
```

2.3.3 SAM Tools

For the SAM [37] (Sequence Alignment Map) tools, download version 0.1.19 from <http://sourceforge.net/projects/samtools/files/> to the *Downloads_tc* folder as a tarball, and unpack them.

```
$ tar -xvjf samtools-0.1.19.tar.bz2
```

The SAM tools are installed by first changing to the unpacked directory and then using the make command to compile the source code. To install SAM tools, one needs to have zlib and htlib

installed. The resultant ‘samtools’ binary is copied into the bin folder added to the PATH.

```
$ make
$ cp samtools $HOME/bin
```

2.3.4 Cufflinks Software

For the demonstration analysis, we have used Cufflinks-2.1.1, which can be downloaded from the Cufflinks website (<http://cufflinks.cbc.b.umd.edu/index.html>). Download the Linux 64-bit binary version to the *Downloads_tc* folder as a tarball and unpack them (*see Note 3*).

```
$ tar -xvzf cufflinks-2.1.1.Linux_x86_64.tar.gz
```

From within the unpacked Cufflinks directory, copy the executable binaries into the bin folder:

```
$ cd cufflinks-2.1.1.Linux_x86_64
$ cp * $HOME/bin
```

2.3.5 CummeRbund Software

The CummeRbund is in R script. To download it, start an R session using the following command:

```
$ R
```

Following which, the documentation for R will appear. Thereafter, type the following commands to install bioconductor and then CummeRbund: (*see Note 4*)

```
>source('http://www.bioconductor.org/biocLite.R')
>biocLite('cummeRbund')
```

3 Methods and Results

3.1 Running TopHat

Type the following commands to map the RNA-seq reads to the genome (*see Note 5*).

```
$ tophat -p 2 -i 20 -I 5000 -G genes.gtf -o SRR671946_
tophatOut genome SRR671946.fastq
```

```
$ tophat -p 2 -i 20 -I 5000 -G genes.gtf -o SRR671947_
tophatOut genome SRR671947.fastq
```

```
$ tophat -p 2 -i 20 -I 5000 -G genes.gtf -o SRR671948_
tophatOut genome SRR671948.fastq
```

```
$ tophat -p 2 -i 20 -I 5000 -G genes.gtf -o SRR671949_
tophatOut genome SRR671949.fastq
```

Here, ‘SRR000000_tophatOut’ represents the output directory for each run. If no output directory is specified by the *-o* option, TopHat automatically creates a directory called *tophat_out* in the working director, and stores the output files in it.

The successful run creates a directory with the name specified by the user, containing the following files: *accepted_hits.bam*, *align_summary.txt*, *deletions.bed*, *insertions.bed*, *junctions.bed*, *prep_reads.info*, *unmapped.bam* and a directory with the name *logs*. The *accepted_hits.bam* is the main result file containing the mapped results in binary format.

3.2 Running Cufflinks to Assemble Transcripts

Using the TopHat alignment files, we can directly run Cufflinks with the following commands to assemble the transcripts (*see Note 6*).

```
$ cufflinks -p 2 -o SRR671946_cufflinksout SRR671946_
tophatOut/accepted_hits.bam
$ cufflinks -p 2 -o SRR671947_cufflinksout SRR671947_
tophatOut/accepted_hits.bam
$ cufflinks -p 2 -o SRR671948_cufflinksout SRR671948_
tophatOut/accepted_hits.bam
$ cufflinks -p 2 -o SRR671949_cufflinksout SRR671949_
tophatOut/accepted_hits.bam
```

For each run, the designated output directory will contain the following files: genes.fpk_tracking, isoforms.fpk_tracking, skipped.gtf, transcripts.gtf. The assembled transcripts are contained in transcripts.gtf.

3.3 Running Cuffmerge to Merge the Transcripts to a Comprehensive Transcriptome

Create a file called assembled_tc.txt that lists out the file path of the assembly file (transcripts.gtf) for each sample. This can be simply done by any text editor, for example nano (*see Note 7*).

```
$ nano
```

Once the text editor is opened, put in the file paths of the four transcript assemblies:

```
./SRR671946_cufflinksout/transcripts.gtf
./SRR671947_cufflinksout/transcripts.gtf
./SRR671948_cufflinksout/transcripts.gtf
./SRR671949_cufflinksout/transcripts.gtf
```

The file is saved using the name ‘assembled_tc.txt’

The assembled transcripts are then used to run Cuffmerge using the following command:

```
$ cuffmerge -g genes.gtf -s genome.fa -p 2 assembled_tc.txt
```

The successful run creates a *merged_asm* directory, which contains a *logs* directory and a file containing the information of the merged transcripts called merged.gtf.

3.4 Running Cuffdiff to Check for Differential Expression Under Different Conditions

Type the following command in a single line (*see Notes 8 and 9*).

```
$ cuffdiff -o diff_result -b genome.fa -p 2 -L Root_Kcl_
control,Root_KNO3_treatment -u merged_asm/merged.gtf ./
SRR671946_tophatOut/accepted_hits.bam,./SRR671947_
tophatOut/accepted_hits.bam,./SRR671948_tophatOut/
accepted_hits.bam,./SRR671949_tophatOut/accepted_hits.bam
```

The successful run creates a directory *diff_result* in the working directory. The directory contains a number of different files and databases, listed as follows:

bias_params.info	cds.diff
cds.count_tracking	cds_exp.diff
cds.fpk_tracking	promoters.diff
cds.read_group_tracking	read_groups.info

gene_exp.diff	run.info
genes.count_tracking	splicing.diff
genes.fpk_tracking	tss_group_exp.diff
genes.read_group_tracking	tss_groups.count_tracking
isoform_exp.diff	tss_groups.fpk_tracking
isoforms.count_tracking	tss_groups.read_group_tracking
isoforms.fpk_tracking	var_model.info
isoforms. read_group_tracking	

The fpkm tracking files give FPKM counts of primary transcripts (tss_groups.fpk_tracking), genes (genes.fpk_tracking), coding sequences (cds.fpk_tracking), and transcripts (isoforms.fpk_tracking).

The count tracking files give the number of fragments for each gene (genes.count_tracking), transcript (isoforms.count_tracking), primary transcript (tss_groups.count_tracking) and coding sequence (cds.count_tracking).

The read group tracking files contain information on the counts of genes, transcripts and primary transcripts, grouped by replicates.

The diff files ending with ‘exp.diff’ contain information on the differential expression tests performed on the genes (gene_exp.diff), primary transcripts (tss_group_exp.diff), transcripts (isoform_exp.diff), and coding sequences (cds_exp.diff).

The promoters.diff file lists out the genes producing two or more primary transcripts, as well as the differential promoter usage of these genes across the samples, in a tabulated format.

The splicing.diff file contains information on primary transcripts that have two or more isoforms, and how they are differently spliced across the samples.

The cds.diff file provides information on the genes that are multi-protein and the extent of their differential coding sequence output between the samples.

Files ending with ‘.info’ contain information mostly on the options provided to and parameter values used by Cuffdiff for expression quantification and assessment of differential expression.

3.5 Results

The original paper from where we have sourced the data [36], used DESeq to analyze the RNA-seq data and highlighted the genes which were differentially expressed across the two conditions. In this section, we show an overall view of the differential gene expression between the two conditions using CummeRbund. Then we use the genes BT1 and ATPPC3 as an example to show

that they are differentially expressed, which is consistent with the findings of the paper [36]. We also use the AT2G33550 gene as an example to show the differential expression of its isoforms across the two conditions, and list out some genes which are similar to the gene BT1. The section also contains some other useful commands for other visualizations like generating scatter and volcano plots.

3.5.1 Running CummeRbund to Generate Visual Graphs and Publishable Figures

Start an R session in the working directory and load the cummeRbund module using the following command:

```
>library('cummeRbund')
```

The readCufflinks function in cummeRbund requires the Cuffdiff output folder, *diff_result*, as an input argument. The following will store the results in the 'cuffdata' object (*see Note 10*).

```
>cuffdata<- readCufflinks('diff_result')
```

```
>cuffdata
```

The last command prints out the Cuffdiff result summary as follows:

CuffSet instance with:

2 samples

33318 genes

42109 isoforms

34957 TSS

32921 CDS

33318 promoters

34957 splicing

27174 relCDS

Plotting Distribution of Expression Levels for Each Sample

To obtain a density plot showing the expression levels for each sample, use the csDensity function:

```
> csDensity(genes(cuffdata))
```

This gives the output of a density plot as an image file (*see Note 11*) (*see Fig. 1*).

Examining Differentially Expressed Genes Across the Conditions Using a Volcano Plot

To obtain a volcano plot showing the differentially expressed genes across the two samples type the following command:

```
>csVolcano(genes(cuffdata), 'Root_Kcl_control', 'Root_KNO3_
treatment')
```

(*See Fig. 2*)

Comparing the Expression of Genes in Each Condition Using a Scatter Plot

To obtain a scatter plot comparing the gene across the two samples type the following command.

```
>csScatter(genes(cuffdata), 'Root_Kcl_control', 'Root_KNO3_
treatment')
```

(*See Fig. 3*)

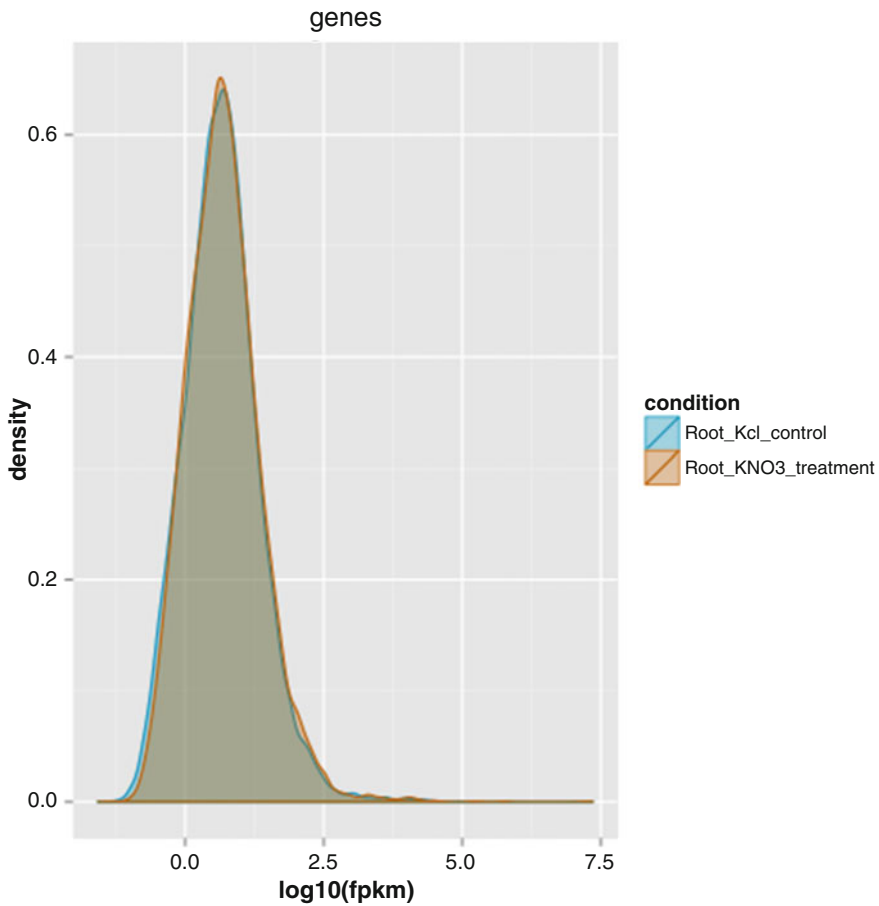


Fig. 1 Density plot of the genes of *Arabidopsis thaliana* root under treated (2 h of 5 mM KNO₃) and control (2 h of 5 mM KCl) conditions

Determining the Number
of Differentially
Expressed Genes

To find the number of differentially expressed genes, type the following command

```
> gene_diff_data <- diffData(genes(cuffdata))
> sig_gene_data <- subset(gene_diff_data, (significant ==
'yes'))
> nrow(sig_gene_data)
[1] 196
```

Recording the Differentially
Expressed Genes' Details

To print a table displaying the details of all the differentially expressed genes, type out the following command.

```
> gene_diff_data <- diffData(genes(cuffdata))
> sig_gene_data <- subset(gene_diff_data, (significant ==
'yes'))
> nrow(sig_gene_data)
```

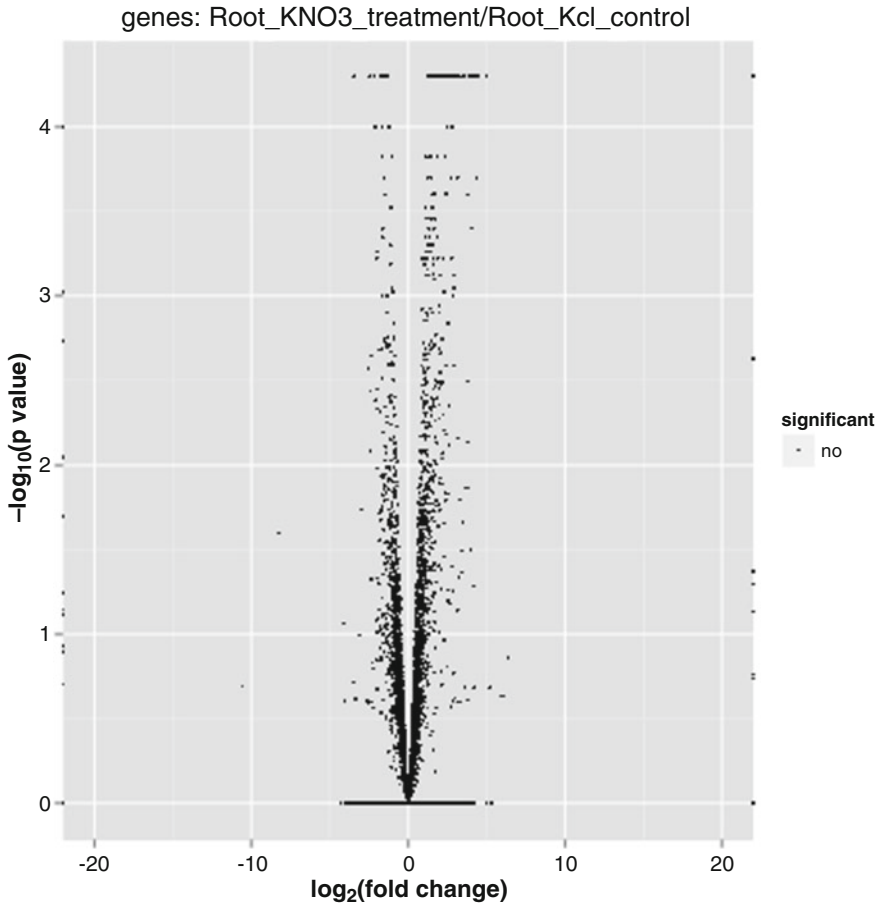


Fig. 2 A volcano plot indicating the presence of differentially expressed genes between the two samples. The plot shows that some genes show a huge change in expression when *Arabidopsis thaliana* root is treated with KNO_3

```
> write.table(sig_gene_data, 'diff_genes.txt', sep = '/t', row.names = F, col.names = T, quote = F)
```

```
> sig_gene_data
```

The last command prints out a table containing the details of all the differentially expressed genes.

3.5.2 Comparing the Expression Levels of Select Genes Using Bar Plots

Compare the gene expression levels of the BT1 gene under both conditions using the following command (*see* Fig. 4).

```
> gene_int <- getGene(cuffdata, 'BT1')
> expressionBarplot(gene_int)
```

Compare the gene expression levels of the ATPPC3 gene under both conditions using the following command (*see* Fig. 5).

```
> gene_int2 <- getGene(cuffdata, 'ATPPC3')
> expressionBarplot(gene_int2)
```

The observations obtained are consistent with the findings in the paper [36], where the authors have used DESeq to analyze the relative expression levels of the differentially expressed genes.

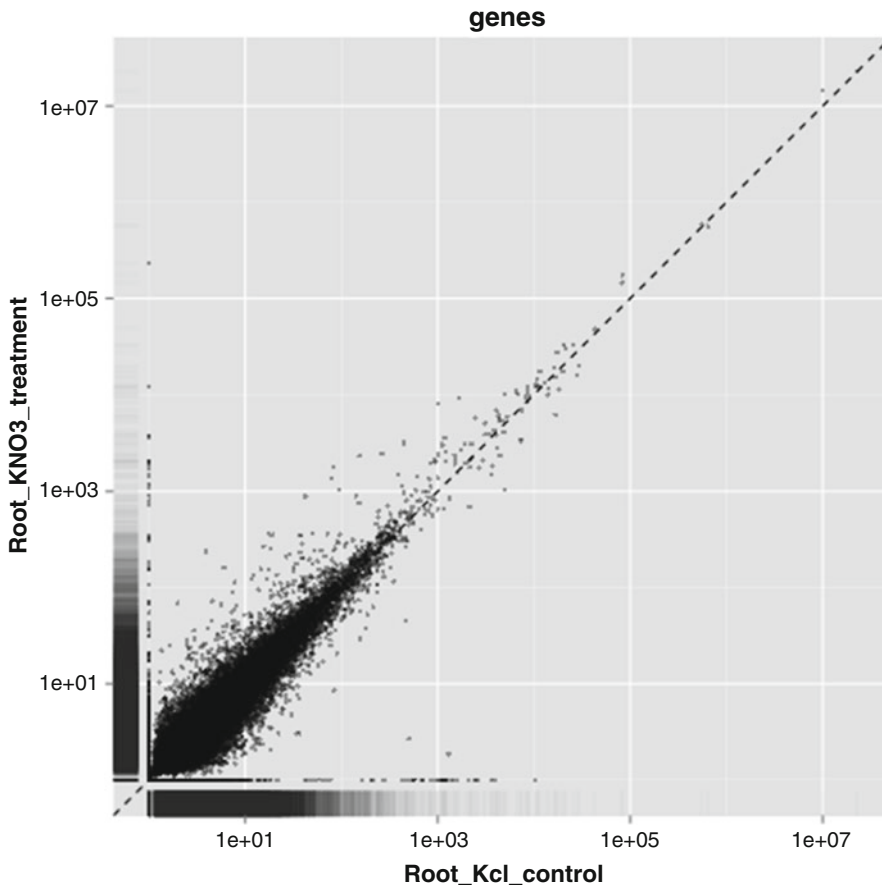


Fig. 3 A scatter plot showing the expression of genes under the two experimental conditions, with the x-axis representing the gene expression values for the control condition and the y-axis representing the gene expression values for the treated condition. Each point thus represents the expression of a gene under both conditions. From the plot, it is evident that that gene expression of some of the genes is increased in the treated condition

3.5.3 Comparing the Expression Levels of the Isoforms of Select Genes Using Bar Plots

Compare the expression levels of the isoforms of the AT2G33550 gene using the following command (*see* Fig. 6a, b).

```
> gene_int3 <- getGene(cuffdata, 'AT2G33550')
> expressionBarplot(isoforms(gene_int3))
```

It is interesting to note how the isoform expression levels roughly add up to give the total gene expression level for a given condition.

3.5.4 Finding Genes Similar to a Given Gene

To find four genes similar to the gene BT1, type out the following command (*see* Fig. 7).

```
> sim_gene <- findSimilar(cuffdata, "BT1", n=4)
> sim_gene.expression <- expressionPlot(sim_gene, log-
Mode=T, showErrorbars=F)
> sim_gene.expression
```

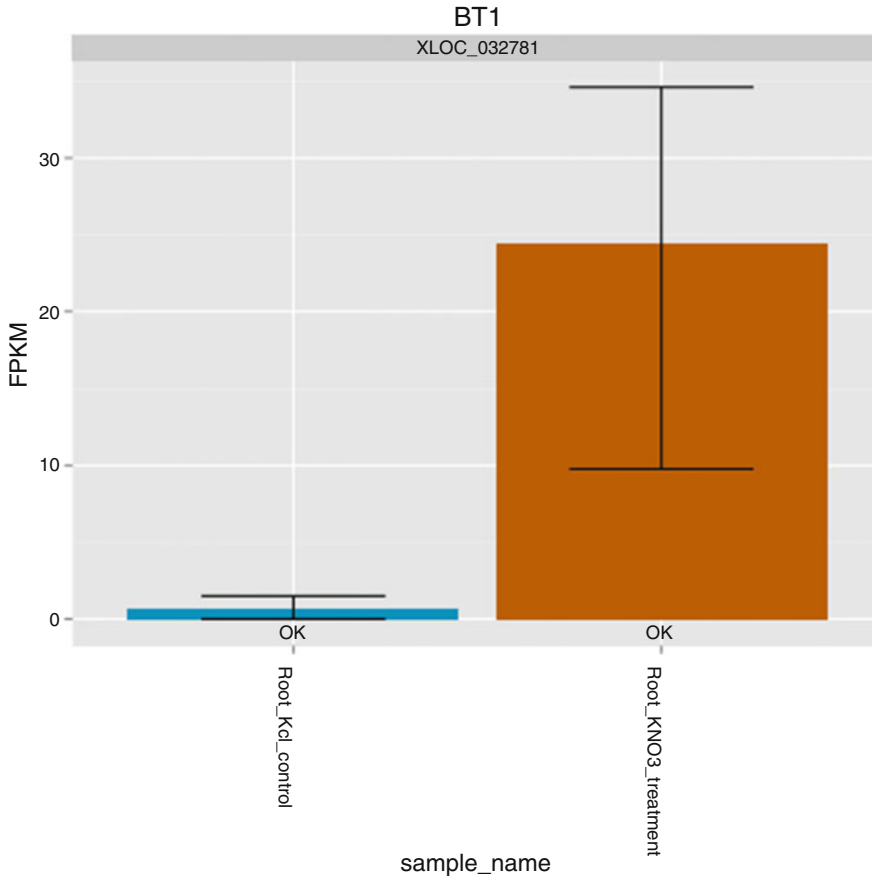


Fig. 4 A bar plot showing the expression of the gene *BT1* under treated (*orange*) and untreated (*blue*) conditions. The FPKM value shown on the *y*-axis is directly proportional to the level of gene expression, so the fold change can be estimated from the graph

4 Notes

1. An easy way to locate the data would be to go to the NCBI website (<http://www.ncbi.nlm.nih.gov>) and to search for the accession number “GSE44062” in the GEO Dataset database. On the search results that show up, click on the result that says “Analysis of the nitrate-responsive transcriptome of the Arabidopsis root”. In the sample section, click on the desired sample, and the sample page will appear with the ftp download link at the bottom of the page. The detailed information about each sample is available in the SRA run archive (http://www.ncbi.nlm.nih.gov/Traces/sra/sra.cgi?view=run_browser) and can be accessed by typing in the run accession for the sample (‘SRR000000’), and then referring to the metadata tab.
2. Although TopHat and Cufflinks make only a few assumptions about the protocol for library preparation for obtaining RNA-

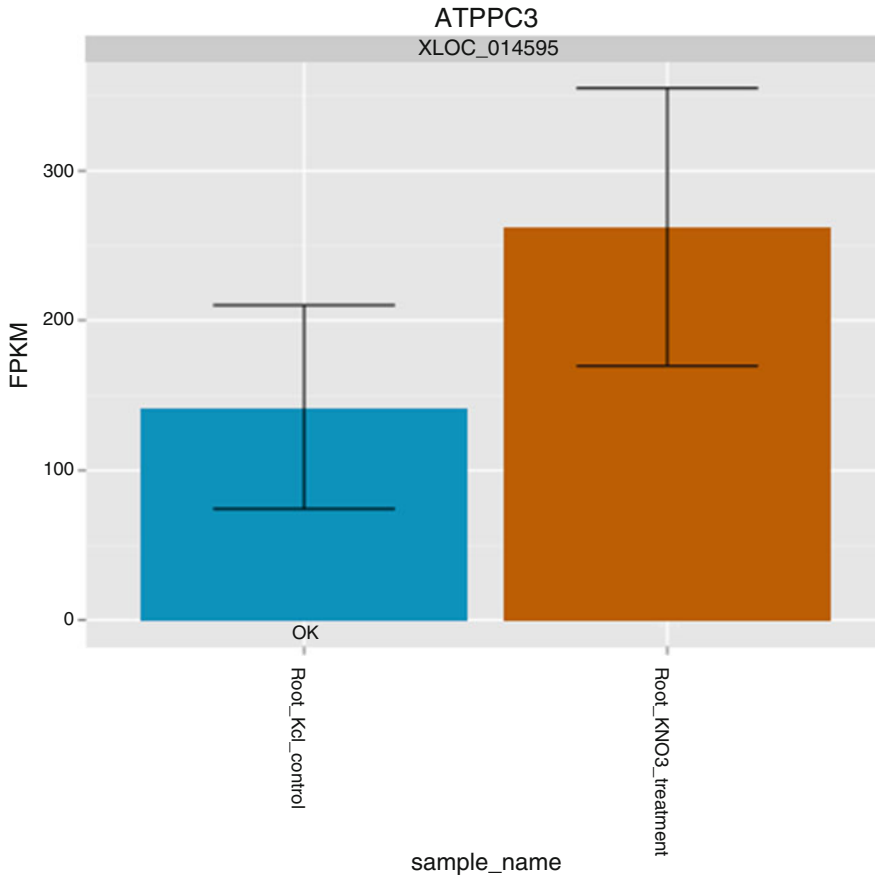


Fig. 5 A bar plot showing the expression of the gene *ATPPC3* under treated (orange) and untreated (blue) conditions. The average expression levels almost correspond to the pattern indicated in the paper, but with different error bars

seq reads, it is very important to note that certain design aspects of the experiment must be considered in order to eliminate problems in mapping reads and estimating transcript abundance. One of these aspects is to have at least three biological replicates for each condition in order to account for biological variability. Another aspect would be a preference for paired end sequencing instead of single end sequencing, in spite of the higher costs, as TopHat works best with 75 bp reads or longer (up to 1024 bp) from single or paired end fragments. Such conditions considerably improve the alignment and splice junction discovery performance for TopHat, as well as the abundance estimates for Cufflinks.

3. To verify if all the tools were installed correctly, users can visit the following pages and refer to their instructions in the “Testing the Installation” section:

<http://cufflinks.cbcb.umd.edu/tutorial.html>

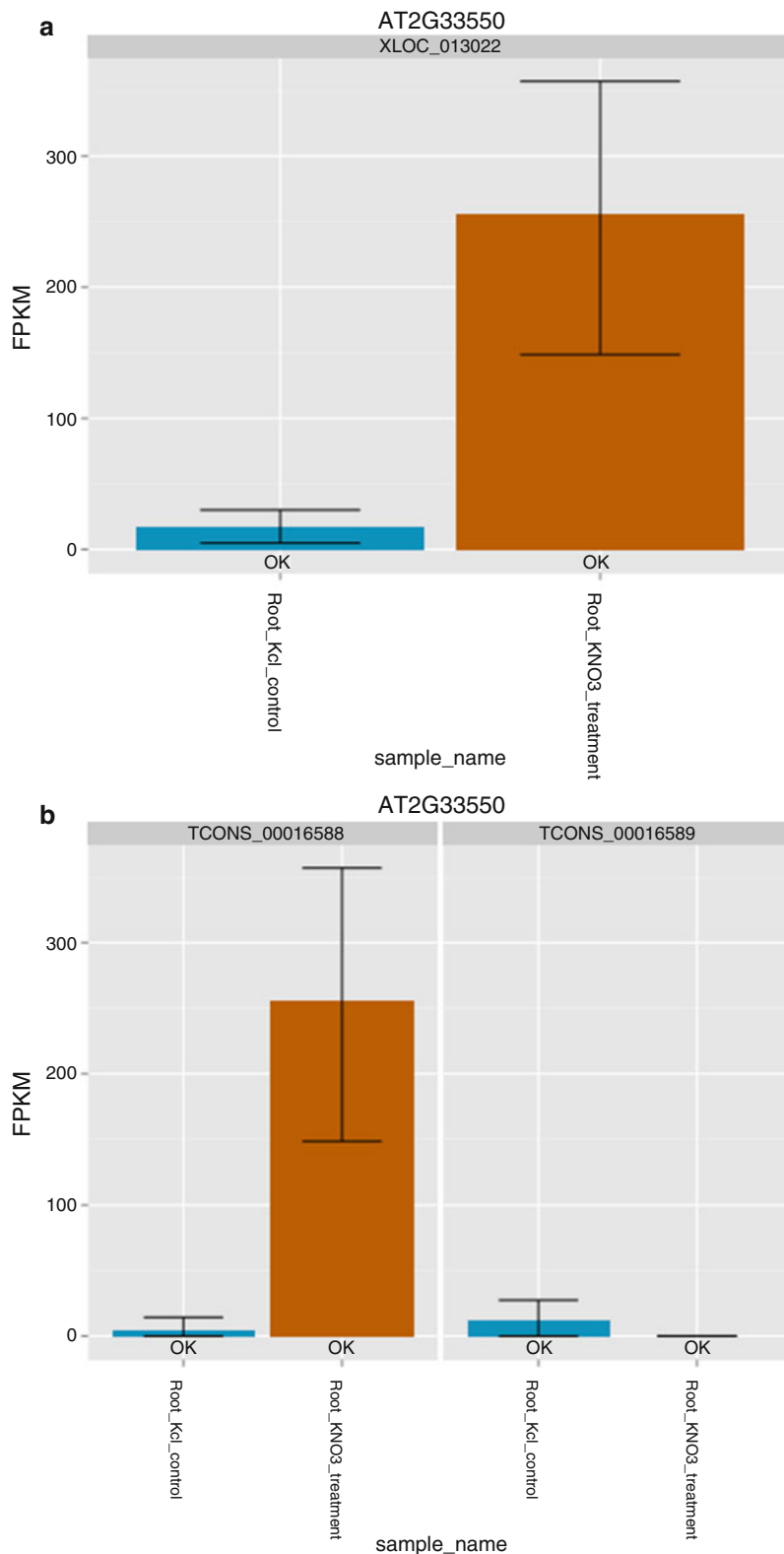


Fig. 6 (a) A bar plot showing the expression of the gene *AT2G33550* under both treated (*orange*) and untreated (*blue*) conditions. **(b)** Bar plots showing the expression levels of the isoforms of the gene *AT2G33550* under both treated (*orange*) and untreated (*blue*) conditions. Note how the isoform FPKM values simply add up to give the total gene expression values for **(a)**

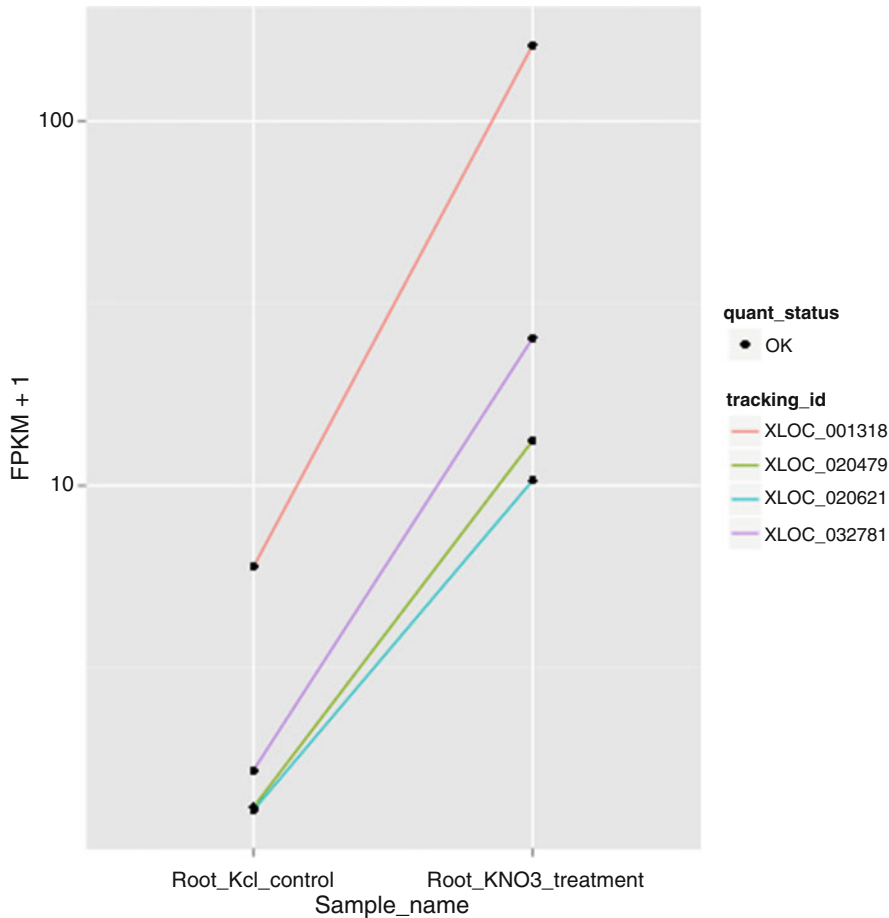


Fig. 7 An illustration output indicating genes that have similar expression patterns to the *BT1* gene

<http://tophat.cbcb.umd.edu/tutorial.shtml>

4. It is very important to note that the CummeRbund package depends on a number of other packages like ggplot2 and RSQLite. A complete list of these packages can be found in the cummeRbund website in the Requirements section (http://compbio.mit.edu/cummeRbund/manual_2_0.html). Most of these packages will automatically be installed as dependencies when the biocLite ('cummeRbund') command is passed. Sometimes, it is advisable to type the command biocLite() before installing the cummeRbund package. To get the best results, install the latest version of R and all the dependent packages.
5. The general command structure for running TopHat with single reads is:


```
$ tophat [options] <genome_index_base> <Single_reads1.fq, Single_reads2.fq...>
```

In the example provided, ‘genome’ is the base name of the genome index. The `-i` and `-I` options are for specifying the minimum and maximum intron length respectively (in base pairs). The values of the intron lengths are adjusted as 20 bp and 5000 bp respectively for *Arabidopsis thaliana* [38, 39] instead of the default values which are for mammals. The `-G` option specifies the annotation file as a guide for read mapping. The `-p` option specifies the number of threads to be used for running. We have used single reads instead of paired-end reads. For paired end reads, each end of a pair must be separately included and added pairwise serially as shown below:

```
$ tophat [options] <genome_index_base> <[PEreads1_1.fq,
PEreads2_1.fq..]> <[PEreads1_2.fq, PEreads2_2.fq...]>
```

TopHat version 2.0.10 allows mixing the FASTQ and FASTA file formats. It also allows the input of both single and paired end reads together in one command. The general command is:

```
$ tophat [options]* <genome_index_base> <[PEreads1_1.fq,
PEreads2_1.fq.., SEreads.fq]> <[PEreads1_2.fq,
PEreads2_2.fq..]>
```

Or

```
$ tophat [options]* <genome_index_base> <[PEreads1_1.fq,
PEreads2_1.fq..]> <[PEreads1_2.fq, PEreads2_2.fq..,
SEreads.fq]>
```

However, it is not recommended to mix different file types or different types of reads in a single run. In order to run it like this, TopHat has a special protocol advice given in the TopHat manual.

Removing the `-G` option, allows users to quantify the samples without any reference annotation.

In order to quantify the reference annotation only, users need to add the `--no-novel-juncs` option to turn off novel transcript discovery.

For colorspace reads, one has to add the `--bowtie1` option as bowtie2 does not support colorspace. (TopHat assumes that the reads are from an Illumina or SOLiD sequencing machine.)

It is also important to note that TopHat has parameters to specify if the library type is strand specific or not. The default setting is for non-strand specific reads, but for strand specific reads the `--library-type` parameter has to be set as ‘fr-firststrand’ or ‘fr-secondstrand’ according to the protocol followed.

The more advanced options include detection of fusion transcripts using the `--fusion-search` option, and also for using annotation data supplied from another source.

For a comprehensive listing and explanation of all the options available for TopHat, please refer to the TopHat manual using the `tophat --help` command or the manual section of the TopHat website (<http://tophat.cbc.umd.edu/manual.shtml>).

6. The general command line structure for running Cufflinks is:
`$ cufflinks [options] <accepted_hits.sam/bam>`

Here the `-o` option specifies the output directory for the written files. The TopHat run produces a file called “accepted_hits.bam”. A BAM file is a compressed binary version of SAM. The latter is a read alignment format that can be used very flexibly in many ways. The SAM tools are installed for access and manipulation of the BAM files. Cufflinks uses this output file to assemble the transcripts and isoforms. Like TopHat, Cufflinks has many options which are out of the scope of this example but can be accessed and viewed in the Cufflinks manual (<http://cufflinks.cbc.umd.edu/manual.html#cufflinks>).

7. The output folder of each run of Cufflinks has a “transcripts.gtf” file, containing information about the assembled transcripts in a tabulated format. The names of each column, their meanings and examples are given in http://cufflinks.cbc.umd.edu/manual.html#cufflinks_output. These “transcript.gtf” files of each sample and condition are pooled in together in the `assembled_tc.txt` file and Cuffmerge uses this file to search out each transcript file and merge it with the others.

Cuffmerge also performs the functions of Cuffcompare when it runs—Cuffcompare simply compares the transcripts that have been assembled through Cufflinks to a reference annotation file (`.gtf`). Cuffmerge also filters out the artifactual transfrags.

The general command structure for Cuffmerge is:

```
$ cuffmerge [options] <assembly_GTF_list.txt>
```

The `-g` option provides Cuffmerge with a reference annotation file and the `-s` option provides it with a genome sequence file in FASTA format. The provision of the reference annotation file leads to merging of the transcripts and isoforms from the sample with known isoforms and transcripts in the reference, thereby providing a better quality of assembly.

One can run Cuffcompare before running Cuffmerge. For more details on the options provided by Cuffcompare, please refer to <http://cufflinks.cbc.umd.edu/manual.html#cuffcompare>.

8. Cufflinks estimates transcript abundances from read counts using the FPKM method. During library construction, a specific narrow size range for reads is selected. This means that a transcript twice as long as another will have twice as many reads when their abundances are almost identical. In order to

avoid any kind of misinterpretation with regard to abundance of each from such data, a normalization is performed on the read count of the fragments by dividing it by the total length of the transcript. In another scenario, it is possible that different experiment runs will produce different amounts of reads. So the same transcript will have different abundances in each run. To avoid such kinds of misinterpretations, another normalization step is involved in which the normalized fragment counts are further divided by the unit total fragment hits. Thus, FPKM (or RPKM), short for fragments (or reads) per kilobase of transcript per million mapped fragments is used which accounts for both the normalization steps. This also makes calculation of gene expression levels easier: Cufflinks calculates abundance estimates for each isoform. So for gene expression determination, one simply needs to add up the FPKM values of each isoform, as FPKM is directly proportional to abundance. In fact, this is also how abundances of transcripts sharing a particular TSS or promoter can be calculated.

9. Cuffdiff calculates differential expression of genes and transcripts across various conditions or time points.

The general command structure for Cuffdiff is:

```
$ cuffdiff [options] <transcripts.gtf> <comma_separated_list_of_all_the_accepted_hits.bam_files_in_the_tophat_output_of_each_replicate_for_condition_1> <comma_separated_list_of_all_the_accepted_hits.bam_files_in_the_tophat_output_of_each_replicate_for_condition_2> ... <..condition N>
```

In this example, the `-o` option specifies the output directory where the Cuffdiff analysis output is to be stored (*diff_result*). The `-p` option specifies the number of threads to be used and the `-b` option specifies that Cuffdiff should use the genome.fa file to make corrections for biases and improve abundance estimate accuracies. The `-u` option is an instruction to accurately weight the reads that map to multiple locations in the genome. The `-L` options specifies the labels that Cuffdiff has to assign to each sample condition in the arguments. Cuffdiff requires a GTF file, produced by Cuffcompare or Cuffmerge (or any other source where the chromosome names must match those in the SAM/BAM alignment files provided) as an argument. The second argument is the alignment files for each sample. Replicates of each sample should be included in the sample argument as comma separated values.

Cuffdiff output contains a set of files containing various information about the differential splicing and expression events across the samples.

For more advanced options and details of Cuffdiff output files, please refer to the Cuffdiff manual in the Cufflinks website (<http://cufflinks.cbcb.umd.edu/manual.html#cuffdiff>).

10. You are likely to get an error in the creation of the 'cuffdata' object. In such case it is better that the hashes be removed from the merged.gtf file produced by Cuffmerge before running Cuffdiff on it.
11. The plots usually are all recorded in a file called Rplots.pdf in the working directory. Often this is not convenient for users. In which case it is better for users to follow the given set of commands:

```
>pdf(file = 'file_name.')
>command_for_creating_visualization
>dev.off()
```

For example, if you were to save a density plot as a pdf with a particular file name, you would type the following commands:

```
>pdf(file = 'scatterplot.pdf')
>csScatter(genes(cuffdata), 'C1', 'C2')
>dev.off()
```

Alternatively, you can save the images in another format like tiff in the following way:

```
>tiff(file = 'file_name.tiff')
>command_for_plot
>dev.off()
```

References

1. Ozsolak F, Milos PM (2011) RNA sequencing: advances, challenges and opportunities. *Nat Rev Genet* 12(2):87–98. doi:[10.1038/nrg2934](https://doi.org/10.1038/nrg2934)
2. Metzker ML (2010) Sequencing technologies - the next generation. *Nat Rev Genet* 11(1):31–46. doi:[10.1038/nrg2626](https://doi.org/10.1038/nrg2626)
3. Mardis ER (2008) The impact of next-generation sequencing technology on genetics. *Trends Genet* 24(3):133–141. doi:[10.1016/j.tig.2007.12.007](https://doi.org/10.1016/j.tig.2007.12.007)
4. Marioni JC, Mason CE, Mane SM, Stephens M, Gilad Y (2008) RNA-seq: an assessment of technical reproducibility and comparison with gene expression arrays. *Genome Res* 18(9):1509–1517. doi:[10.1101/gr.079558.108](https://doi.org/10.1101/gr.079558.108)
5. Ozsolak F, Platt AR, Jones DR, Reifemberger JG, Sass LE, McInerney P, Thompson JF, Bowers J, Jarosz M, Milos PM (2009) Direct RNA sequencing. *Nature* 461(7265):814–818. doi:[10.1038/nature08390](https://doi.org/10.1038/nature08390)
6. Roy SW, Irimia M (2008) When good transcripts go bad: artifactual RT-PCR 'splicing' and genome analysis. *BioEssays* 30(6):601–605. doi:[10.1002/bies.20749](https://doi.org/10.1002/bies.20749)
7. Garber M, Grabherr MG, Guttman M, Trapnell C (2011) Computational methods for transcriptome annotation and quantification using RNA-seq. *Nat Methods* 8(6):469–477. doi:[10.1038/nmeth.1613](https://doi.org/10.1038/nmeth.1613)
8. Trapnell C, Williams BA, Pertea G, Mortazavi A, Kwan G, van Baren MJ, Salzberg SL, Wold BJ, Pachter L (2010) Transcript assembly and

- quantification by RNA-Seq reveals unannotated transcripts and isoform switching during cell differentiation. *Nat Biotechnol* 28(5):511–515. doi:[10.1038/nbt.1621](https://doi.org/10.1038/nbt.1621)
9. Wang Z, Gerstein M, Snyder M (2009) RNA-Seq: a revolutionary tool for transcriptomics. *Nat Rev Genet* 10(1):57–63. doi:[10.1038/nrg2484](https://doi.org/10.1038/nrg2484)
 10. Schena M, Shalon D, Davis RW, Brown PO (1995) Quantitative monitoring of gene expression patterns with a complementary DNA microarray. *Science* 270(5235):467–470
 11. Schena M, Shalon D, Heller R, Chai A, Brown PO, Davis RW (1996) Parallel human genome analysis: microarray-based expression monitoring of 1000 genes. *Proc Natl Acad Sci U S A* 93(20):10614–10619
 12. Shiraki T, Kondo S, Katayama S, Waki K, Kasukawa T, Kawaji H, Kodzius R, Watahiki A, Nakamura M, Arakawa T, Fukuda S, Sasaki D, Podhajski A, Harbers M, Kawai J, Carninci P, Hayashizaki Y (2003) Cap analysis gene expression for high-throughput analysis of transcriptional starting point and identification of promoter usage. *Proc Natl Acad Sci U S A* 100(26):15776–15781. doi:[10.1073/pnas.2136655100](https://doi.org/10.1073/pnas.2136655100)
 13. Kodzius R, Kojima M, Nishiyori H, Nakamura M, Fukuda S, Tagami M, Sasaki D, Imamura K, Kai C, Harbers M, Hayashizaki Y, Carninci P (2006) CAGE: cap analysis of gene expression. *Nat Methods* 3(3):211–222. doi:[10.1038/nmeth0306-211](https://doi.org/10.1038/nmeth0306-211)
 14. Velculescu VE, Zhang L, Vogelstein B, Kinzler KW (1995) Serial analysis of gene expression. *Science* 270(5235):484–487
 15. Reinartz J, Bruyns E, Lin JZ, Burcham T, Brenner S, Bowen B, Kramer M, Woychik R (2002) Massively parallel signature sequencing (MPSS) as a tool for in-depth quantitative gene expression profiling in all organisms. *Brief Funct Genomic Proteomic* 1(1):95–104
 16. Brenner S, Johnson M, Bridgham J, Golda G, Lloyd DH, Johnson D, Luo S, McCurdy S, Foy M, Ewan M, Roth R, George D, Eletr S, Albrecht G, Vermaas E, Williams SR, Moon K, Burcham T, Pallas M, DuBridge RB, Kirchner J, Fearon K, Mao J, Corcoran K (2000) Gene expression analysis by massively parallel signature sequencing (MPSS) on microbead arrays. *Nat Biotechnol* 18(6):630–634. doi:[10.1038/76469](https://doi.org/10.1038/76469)
 17. Mortazavi A, Williams BA, McCue K, Schaeffer L, Wold B (2008) Mapping and quantifying mammalian transcriptomes by RNA-Seq. *Nat Methods* 5(7):621–628. doi:[10.1038/nmeth.1226](https://doi.org/10.1038/nmeth.1226)
 18. Feldmeyer B, Wheat CW, Krezdorn N, Rotter B, Pfenninger M (2011) Short read Illumina data for the de novo assembly of a non-model snail species transcriptome (*Radix balthica*, Basommatophora, Pulmonata), and a comparison of assembler performance. *BMC Genomics* 12:317. doi:[10.1186/1471-2164-12-317](https://doi.org/10.1186/1471-2164-12-317)
 19. Langmead B, Trapnell C, Pop M, Salzberg SL (2009) Ultrafast and memory-efficient alignment of short DNA sequences to the human genome. *Genome Biol* 10(3):R25. doi:[10.1186/gb-2009-10-3-r25](https://doi.org/10.1186/gb-2009-10-3-r25)
 20. Li H, Durbin R (2010) Fast and accurate long-read alignment with Burrows-Wheeler transform. *Bioinformatics* 26(5):589–595. doi:[10.1093/bioinformatics/btp698](https://doi.org/10.1093/bioinformatics/btp698)
 21. Simpson JT, Durbin R (2010) Efficient construction of an assembly string graph using the FM-index. *Bioinformatics* 26(12):i367–i373. doi:[10.1093/bioinformatics/btq217](https://doi.org/10.1093/bioinformatics/btq217)
 22. Trapnell C, Pachter L, Salzberg SL (2009) TopHat: discovering splice junctions with RNA-Seq. *Bioinformatics* 25(9):1105–1111. doi:[10.1093/bioinformatics/btp120](https://doi.org/10.1093/bioinformatics/btp120)
 23. Trapnell C, Roberts A, Goff L, Pertea G, Kim D, Kelley DR, Pimentel H, Salzberg SL, Rinn JL, Pachter L (2012) Differential gene and transcript expression analysis of RNA-seq experiments with TopHat and Cufflinks. *Nat Protoc* 7(3):562–578. doi:[10.1038/nprot.2012.016](https://doi.org/10.1038/nprot.2012.016)
 24. Goff L, Trapnell C, Kelley D, Guide PRSCU, bioViews Clustering D, DataRepresentation D, GeneExpression I, MultipleComparison Q, RNASeq R, since BioC IB (2012) Analysis, exploration, manipulation, and visualization of Cufflinks high-throughput sequencing data. R package version 2 (1)
 25. Goecks J, Nekrutenko A, Taylor J, Galaxy T (2010) Galaxy: a comprehensive approach for supporting accessible, reproducible, and transparent computational research in the life sciences. *Genome Biol* 11(8):R86. doi:[10.1186/gb-2010-11-8-r86](https://doi.org/10.1186/gb-2010-11-8-r86)
 26. Li R, Yu C, Li Y, Lam TW, Yiu SM, Kristiansen K, Wang J (2009) SOAP2: an improved ultrafast tool for short read alignment. *Bioinformatics* 25(15):1966–1967. doi:[10.1093/bioinformatics/btp336](https://doi.org/10.1093/bioinformatics/btp336)
 27. Lunter G, Goodson M (2011) Stampy: a statistical algorithm for sensitive and fast mapping of Illumina sequence reads. *Genome Res* 21(6):936–939. doi:[10.1101/gr.111120.110](https://doi.org/10.1101/gr.111120.110)
 28. Wang K, Singh D, Zeng Z, Coleman SJ, Huang Y, Savich GL, He X, Mieczkowski P, Grimm SA, Perou CM, MacLeod JN, Chiang DY, Prins JF, Liu J (2010) MapSplice: accurate

- mapping of RNA-seq reads for splice junction discovery. *Nucleic Acids Res* 38(18), e178. doi:[10.1093/nar/gkq622](https://doi.org/10.1093/nar/gkq622)
29. Wu TD, Nacu S (2010) Fast and SNP-tolerant detection of complex variants and splicing in short reads. *Bioinformatics* 26(7):873–881. doi:[10.1093/bioinformatics/btq057](https://doi.org/10.1093/bioinformatics/btq057)
 30. Guttman M, Garber M, Levin JZ, Donaghey J, Robinson J, Adiconis X, Fan L, Koziol MJ, Gnirke A, Nusbaum C, Rinn JL, Lander ES, Regev A (2010) Ab initio reconstruction of cell type-specific transcriptomes in mouse reveals the conserved multi-exonic structure of lincRNAs. *Nat Biotechnol* 28(5):503–510. doi:[10.1038/nbt.1633](https://doi.org/10.1038/nbt.1633)
 31. Zerbino DR, Birney E (2008) Velvet: algorithms for de novo short read assembly using de Bruijn graphs. *Genome Res* 18(5):821–829. doi:[10.1101/gr.074492.107](https://doi.org/10.1101/gr.074492.107)
 32. Lee S, Seo CH, Lim B, Yang JO, Oh J, Kim M, Lee S, Lee B, Kang C, Lee S (2011) Accurate quantification of transcriptome from RNA-Seq data by effective length normalization. *Nucleic Acids Res* 39(2):e9. doi:[10.1093/nar/gkq1015](https://doi.org/10.1093/nar/gkq1015)
 33. Wang L, Feng Z, Wang X, Wang X, Zhang X (2010) DEGseq: an R package for identifying differentially expressed genes from RNA-seq data. *Bioinformatics* 26(1):136–138. doi:[10.1093/bioinformatics/btp612](https://doi.org/10.1093/bioinformatics/btp612)
 34. Anders S, Huber W (2010) Differential expression analysis for sequence count data. *Genome Biol* 11(10):R106. doi:[10.1186/gb-2010-11-10-r106](https://doi.org/10.1186/gb-2010-11-10-r106)
 35. Twine NA, Janitz K, Wilkins MR, Janitz M (2011) Whole transcriptome sequencing reveals gene expression and splicing differences in brain regions affected by Alzheimer's disease. *PLoS One* 6(1), e16266
 36. Vidal EA, Moyano TC, Krouk G, Katari MS, Tanurdzic M, McCombie WR, Coruzzi GM, Gutierrez RA (2013) Integrated RNA-seq and sRNA-seq analysis identifies novel nitrate-responsive genes in *Arabidopsis thaliana* roots. *BMC Genomics* 14:701. doi:[10.1186/1471-2164-14-701](https://doi.org/10.1186/1471-2164-14-701)
 37. Li H, Handsaker B, Wysoker A, Fennell T, Ruan J, Homer N, Marth G, Abecasis G, Durbin R, Genome Project Data Processing S (2009) The Sequence Alignment/Map format and SAMtools. *Bioinformatics* 25(16):2078–2079. doi:[10.1093/bioinformatics/btp352](https://doi.org/10.1093/bioinformatics/btp352)
 38. Alexandrov NN, Troukhan ME, Brover VV, Tatarinova T, Flavell RB, Feldmann KA (2006) Features of *Arabidopsis* genes and genome discovered using full-length cDNAs. *Plant Mol Biol* 60(1):69–85. doi:[10.1007/s11103-005-2564-9](https://doi.org/10.1007/s11103-005-2564-9)
 39. Filichkin SA, Priest HD, Givan SA, Shen R, Bryant DW, Fox SE, Wong WK, Mockler TC (2010) Genome-wide mapping of alternative splicing in *Arabidopsis thaliana*. *Genome Res* 20(1):45–58. doi:[10.1101/gr.093302.109](https://doi.org/10.1101/gr.093302.109)